Lab 5
Low Complexity Model

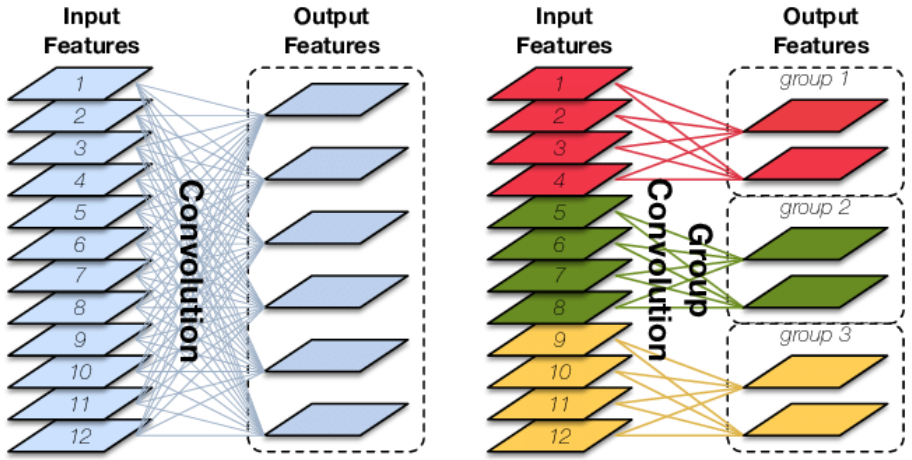
Why do we need low-complexity model

- Reduce execution time
  - Include computation time and memory access time
- Reduce parameters
  - Reduce memory access
- Reduce FLOPs
  - Reduce computation time

- **Objective in this Lab: fewer parameters, lower FLOPs, maintain accuracy**

Decomposition

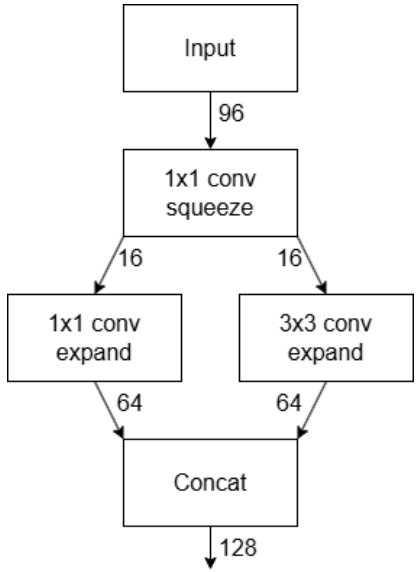
- Convolution has high computational and memory demands.
  - Decomposition can reduce both FLOPs and parameters
- Group convolution
  - Example: ResNeXt
- Separable convolution
  - Depth-wise convolution + point-wise convolution
  - Example: MobileNet, Xception, EfficientNet



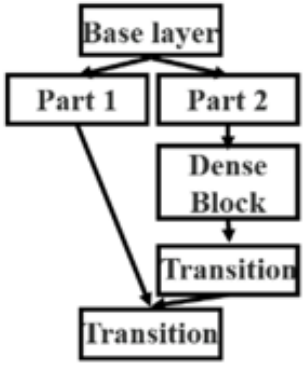
Channel reduction and partition

- Reducing the channel might also be a good method to reduce parameters and FLOPs

- Channel reduce
  - Reduce the channel by performing 1x1 convolution
  - Example: SqueezeNet



- Channel partition
  - Calculate only on part of the channel, and then concatenate the result
  - Example: CSPNet



(b) CSPDenseNet

Other methods

- The method used in Lab 4 can also be effectively applied in this lab, alongside the approaches mentioned above.
  - Pruning
  - Quantization

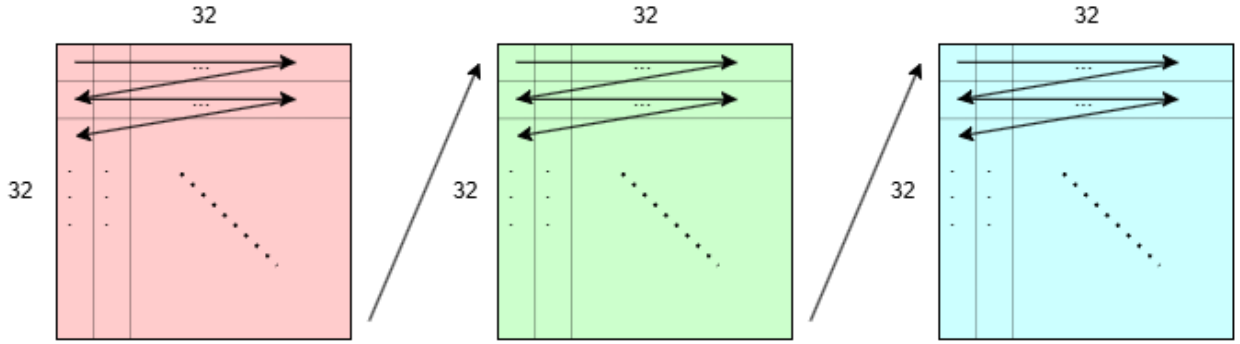
Accuracy Restore

- The former methods might cause a drop in accuracy. How could we restore it?
  - create or reuse gradient diversity as much as possible

- Scale input size/depth/width
- Multiple gradient path
  - Example: Dense layer in DenseNet
- Channel Shuffle
  - Example: ShuffleNet
- Attention
  - Example: Squeeze-Excitation network in SqueezeNet

Dataset – Cifar100

- Image classification
- 32 x 32 pixels each image
- 60000 images
  - 40000 training images
  - 10000 validation images
  - 10000 test images (Hidden)



Input size : 3(channels) x 32(pixels) x 32(pixels) each image

Superclass	Classes
aquatic mammals	beaver, dolphin, otter, seal, whale
fish	aquarium fish, flatfish, ray, shark, trout
flowers	orchids, poppies, roses, sunflowers, tulips
food containers	bottles, bowls, cans, cups, plates
fruit and vegetables	apples, mushrooms, oranges, pears, sweet peppers
household electrical devices	clock, computer keyboard, lamp, telephone, television
household furniture	bed, chair, couch, table, wardrobe
insects	bee, beetle, butterfly, caterpillar, cockroach
large carnivores	bear, leopard, lion, tiger, wolf
large man-made outdoor things	bridge, castle, house, road, skyscraper
large natural outdoor scenes	cloud, forest, mountain, plain, sea
large omnivores and herbivores	camel, cattle, chimpanzee, elephant, kangaroo
medium-sized mammals	fox, porcupine, possum, raccoon, skunk
non-insect invertebrates	crab, lobster, snail, spider, worm
people	baby, boy, girl, man, woman
reptiles	crocodile, dinosaur, lizard, snake, turtle
small mammals	hamster, mouse, rabbit, shrew, squirrel
trees	maple, oak, palm, pine, willow
vehicles 1	bicycle, bus, motorcycle, pickup truck, train
vehicles 2	lawn-mower, rocket, streetcar, tank, tractor

Task (Lab5.ipynb)

- Step 1：load dataset
  - The batch size values for the train and valid could be set whatever you want, and they could be different.

```
train_set = torch.load("cifar100/train.pt")
valid_set = torch.load("cifar100/valid.pt")

batch_size = 32
train_loader = DataLoader(train_set, batch_size, shuffle=False)
valid_loader = DataLoader(valid_set, batch_size, shuffle=False)
```

Task (Lab5.ipynb)

- Step 2：
  - Select the desired model architecture as backbone
  - Build and adjust it on **network.py**
  - **Any method** can be used as long as it effectively reduces model complexity.

```python
import torch

### Write your model architecture

### End

def load_model(MODEL_PATH):
    #call model
    model =
    model = torch.load(MODEL_PATH)
    return model
```

  - Call the model on jupyter notebook

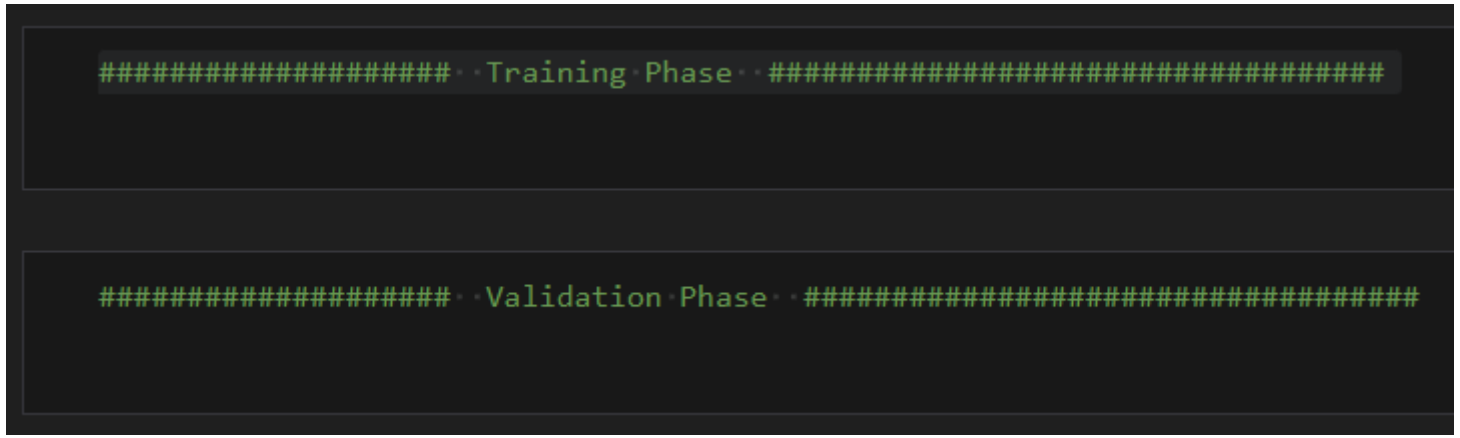
```python
# using gpu if available
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

#build your model here

#call model
model =
```

Task (Lab5.ipynb)

- Write both training and validation phase by yourself



- Save your best model as **.pth** file after the process
- **Only .pth files are accepted. Submitting other types of files will result in a zero grade.**

Testing flow

- You need to prepare
  - network.py： your model architecture
  - model.pth：saved model file
- Command
  - python test.py <DATA_PATH> <MODEL_PATH>
- Example output

```
flops = 37266432
Model parameter size = 43896.453 kB
Accuracy = 80.27 %
```

- Please make sure you can successfully run the test.py and have the results.

- **Do not modify the test.py!**

Report should contain

- Which methods do you apply to lower complexity and bridge the accuracy gap
- Why do you apply the methods
- How do you implement
- Screenshot of your testing results
  - FLOPs
  - Model size
  - Accuracy

Grading Policy

- Three requirements are fulfilled **(30 %)**
  - < 0.5 GFLOPs
  - < 20 MB
  - Validation accuracy > 80 % (top-1 acc)
- Report **(30 %)**
- Performance **(40 %)**
  - $FoM = \frac{1}{(FLOPs)^2} \times \frac{1}{model\ size^2} \times acc.$
  - Acc. in performance will be determined based on a hidden test set
  - The larger, the better

Reminder

- Submit Deadline : 2 weeks (2024/11/25 23:59)
- Please strictly follow the naming rules !!!
- Upload 4 files to new e3
  - Lab05.ipynb
  - network.py
  - model.pth
  - StudentID_report.pdf (example: 311555555_report.pdf)

Have Fun !!!!!!